

Sliding Puzzle Project — Development Log

Day 5 — Wednesday, August 12, 2026

Goal for the project

Build four Python puzzle projects (a playable sliding tile puzzle, a solver for it, a playable 3D Rubik's cube, and a solver for that) as a learning exercise. The user writes all code themselves; guidance is conceptual, not code handed over directly.

Today's focus

Finishing Stage 4 (centring tile numbers) and completing Stage 5 (input handling) — wiring both arrow-key and mouse-click input into `render.py`, routed through the existing `apply_move()` logic rather than duplicating movement rules.

What happened

- Finished centring tile numbers by offsetting the blit position by half the tile size in both directions ($x + \text{tile_size} // 2$, $y + \text{tile_size} // 2$), replacing the earlier top-left-anchored placement. Flagged as an approximation that works well for single-digit tiles but won't perfectly centre wider multi-character numbers (e.g. "11") without also accounting for the rendered text's own width/height — left as a known limitation, not fixed today.
- Designed the keyboard-to-move mapping before writing any code: pressing an arrow key moves the tile the arrow points toward into the blank space (e.g. up arrow moves the tile below the blank upward). Derived all four target positions relative to the blank's position ($r0$, $c0$): up $\rightarrow (r0+1, c0)$, down $\rightarrow (r0-1, c0)$, left/right $\rightarrow (r0, c0\mp 1)$. During implementation, left/right were initially swapped from the derived rule; tested directly against actual on-screen behaviour and corrected to what was confirmed to feel right by testing, rather than trusting the abstract derivation alone.
- Wired KEYDOWN handling into `render.py`'s existing event loop: for each arrow key, computed the target (row, col) and called `movement_logic.apply_move(dictionary, move)` directly inside that key's branch (rather than once after the whole if/elif chain), specifically to avoid an `UnboundLocalError` if a non-arrow key were pressed and move were never assigned that cycle.
- Built `pixel_to_tile(x, y)` to convert a mouse click's raw pixel position into a (row, col) board position, using integer division (`row = y // tile_size`, `column = x // tile_size`) — the reverse of the row/col-to-pixel conversion used for drawing. Caught and fixed a missing return statement on the first attempt (the same category of bug as the very first bug of the whole project — a function that computes a value but never hands it back to its caller).
- Worked through a design decision on how `pixel_to_tile` should get `tile_size`: whether to declare it as a parameter (self-contained, but must be supplied at every call) or rely on it being read from the surrounding `render.py` scope (fewer arguments needed at the call site, but the function becomes silently dependent on a variable existing nearby). Chose not to declare it as a parameter, relying on `render.py`'s existing `tile_size` variable via Python's scope lookup.
- Wired MOUSEBUTTONDOWN handling into the same event loop: reads `event.pos` for the click's pixel coordinates, converts via `pixel_to_tile()`, and calls `apply_move()` the same way as the keyboard path — no movement logic duplicated between the two input methods.

- Confirmed both input paths working: arrow keys and mouse clicks both correctly move only tiles adjacent to the blank, and both fail safely (via `apply_move`'s existing `True/False` + `print` behaviour) when an illegal move is attempted, without corrupting the board state.

Concepts & Lessons Learned (study notes)

A function can read a variable from outside itself, but not reassign it, without extra steps

Python's scope lookup (LEGB — Local, Enclosing, Global, Built-in) means a function can read a variable defined outside it (e.g. `tile_size` at the top of `render.py`) even without receiving it as a parameter, as long as the function never assigns to that name itself. If a function does try to reassign a name that also exists outside it, Python treats that name as local for the function's entire body — which can produce an `UnboundLocalError` on an earlier read, the same category of bug encountered on Day 2. Choosing between "pass it as a parameter" and "let the function read the outer variable" is a real design decision: parameters make a function self-contained and reusable elsewhere; relying on an outer variable is less to type at each call site but ties the function invisibly to wherever it happens to be defined.

Parameters vs. arguments: definition-time placeholders vs. call-time values

`def some_function(a, b, c):` declares placeholder names (parameters) a function needs — nothing runs yet. Calling it, e.g. `some_function(x, y, z)`, supplies the actual values (arguments) to use this time, matched to the parameters by position. If a parameter is named the same as an outer variable being passed in (as happened with `tile_size` here), that's a naming coincidence, not a special case — Python only cares about position, not whether the names match. Once a parameter is declared in a function's definition, it must be supplied at every call, or Python raises a `missing-argument error` — there is no way to "skip" a required parameter just because a same-named variable already exists elsewhere, unless the parameter is given a default value in the definition itself.

Converting pixels back to board coordinates is the reverse of the drawing conversion

Drawing converts (row, col) to pixels via `x = col * tile_size`, `y = row * tile_size`. Reading a click converts the other direction, using integer division to find which tile-sized block a pixel falls into: `row = y // tile_size`, `col = x // tile_size`. Both directions rely on the same `tile_size` and the same top-left-origin, y-increases-downward convention established when rendering was first built.

Testing observed behaviour can override an abstract derivation

The left/right key mapping was worked out on paper first, then found to be backwards once actually run and watched on screen; it was corrected to match what was confirmed correct by direct observation, rather than re-deriving or trusting the earlier logic on faith. General lesson: a derivation worked out in the abstract is a good starting hypothesis, but running the actual code and watching the real result is the final check — especially for anything involving direction, order, or orientation, where it's easy for a sign or an axis to end up flipped somewhere in the reasoning without an error ever being raised.

Status at end of day

Stage 4 (rendering) is fully complete, including centred tile numbers (with a known minor limitation on multi-digit tiles). Stage 5 (input handling) is fully complete: both arrow-key and mouse-click input are wired into `render.py`'s event loop and correctly delegate all movement rules to `movement_logic.apply_move()`, with no duplicated logic and safe handling of illegal move attempts.

Next up

Stage 6 — game feel: a move counter (incrementing on successful moves, using `apply_move`'s `True/False` return value to distinguish a real move from a rejected one), a timer starting on shuffle and stopping when `is_solved()` becomes true, a "Solved!" message, and a key to trigger a reshuffle mid-game.